ITSS Workshop – Participant Handout

Agentic AI for Software Engineering | May 28, 2026

Companion cheatsheets (six printable PDFs in this Drive folder alongside the handout):

- `01_CLAUDE_AGENTS_md_Cheatsheet.pdf` – CLAUDE.md / AGENTS.md authoring patterns
- `02_Kill_Signal_Decision_Card.pdf` – CODE / PEOPLE triage for brownfield work
- `03_MCP_Plugin_Supply_Chain_Checklist.pdf` – MCP and plugin install governance
- `04_Repo_Architecture_Mapping_Prompt.pdf` – parallel-subagent prompt for any codebase
- `05_Documentation_Mindmap_Prompt.pdf` – parallel-subagent prompt for any documentation corpus
- `06_Manager_Day7_Worksheet.pdf` – one-page worksheet for the manager track, Day 7 after the workshop

Run on: vendor API docs before integration, Confluence onboarding spaces, regulatory archives, an exported `/docs` folder. Typical: 80-150 documents, 10-20 minutes wall-clock, dual HTML+INDEX output with page citations.

MCP = Model Context Protocol – vendor-neutral integration layer connecting AI agents to external systems (Jira, Confluence, GitHub, databases). Think of it as the JDBC of AI agents.

YOUR STARTER PACK – Install This Week

The `claude-plugins-official` marketplace ships built-in with Claude Code – no `/plugin marketplace add` step needed. The third-party `Lum1104/Understand-Anything` marketplace does require an explicit add. Open a Claude Code session (`claude` in your terminal), then run these **slash commands** inside it (NOT bash):

```
/plugin marketplace add Lum1104/Understand-Anything       # Third-party marketplace (one-time)
/plugin install hookify@claude-plugins-official           # Custom team rules in markdown
/plugin install security-guidance@claude-plugins-official # Vulnerability prevention (8 categories)
/plugin install superpowers@claude-plugins-official       # phase-based delivery methodology
/plugin install pr-review-toolkit@claude-plugins-official # 6-agent code review
/plugin install plugin-dev@claude-plugins-official        # SDK for scaffolding your own plugins + marketplace
/plugin install understand-anything@understand-anything   # Visual codebase graph (Structural + Domain views)
```

The install command shape is always `/plugin install <name>@<marketplace>`. `claude-plugins-official` is built-in; the `understand-anything` marketplace name comes from the one-time `marketplace add` above.

> ***Install fallback:*** *If `claude-plugins-official` is not listed in the marketplace, run `/plugin marketplace add anthropics/claude-plugins-official` first, then retry the install commands above. Source: Anthropic Claude Code plugins documentation.*

Verify: `/plugin list`

Build & Own Your Marketplace (Demo 1 Part 2 take-home)

In Demo 1 Part 2 (Slide 8, live), you saw the `plugin-dev` SDK scaffold a complete plugin marketplace – manifest, plugins directory, one sample plugin with a real working SKILL.md – in about a minute.

> ***Architecture-html plugin*** *– the marketplace demo Mihai scaffolded live takes the Repo Architecture Mapping prompt you have on your cheatsheet, wraps it as a reusable skill, and ships it as a plugin in your internal marketplace. Anyone on the team runs* `/plugin install architecture-html@itss-plugins` *once, then* `architecture-html` *becomes a one-word command on any repo (no need to paste the 80-line prompt). That's "build before you restrict" – the constructive half of governance.*

The point isn't the specific plugin. The point is: **your marketplace is a git repo your team owns and maintains, exactly like any other production repo.**

This is the constructive half of the Permissions / Sandbox primitive you saw on Slide 7 – four configuration surfaces: defaults (settings.json), project overrides, hooks, and OS sandbox. Those surfaces are restrictive guardrails (allow / ask / deny / sandbox). This is what you BUILD inside them.

The marketplace is just a repo

A marketplace is a git repo with two things at its root:

1. A `.claude-plugin-marketplace.json` manifest that lists which plugins live in the repo
2. A `plugins/` directory containing one subdirectory per plugin

That's it. Same git workflow your team already uses. Same PR review. Same semver tags. Same security review. Same `CODEOWNERS`. Same release notes. **Nothing new to operate.**

The official `claude-plugins-official` marketplace ships built-in with Claude Code – you can read its source (it's a public GitHub repo at `anthropics/claude-plugins-official`). Your team's private marketplace is structurally identical, just hosted on your internal git server.

Scaffold your own via the plugin-dev SDK

The `plugin-dev` plugin is in your starter pack (`/plugin install plugin-dev@claude-plugins-official` – already installed if you ran the bootstrap commands above). Inside any Claude Code session:

```
Use the plugin-dev:create-plugin skill to scaffold a Claude Code plugin
MARKETPLACE at ~/work/<your-team>-plugins/ that <YourCompany> would own
internally. Include the marketplace manifest, a plugins/ directory, and
one sample plugin called architecture-html with skills/architecture-html/SKILL.md
wrapping the generic Repo Architecture Mapping prompt (parallel-subagent
exploration of any repo, interactive HTML output with file:// citations).
README.md should explain how to publish + install. Show the resulting
file tree when done.
```

Then `git init`, `git remote add origin <your-internal-git-url>`, push, and your team has a marketplace – and `architecture-html` becomes a one-word command on any repo your team works on.

Curation = governance

The hard part isn't scaffolding plugins. The hard part is keeping a marketplace **standardized at scale**. Apply the same engineering discipline you already use for backend code:

- **PR review per plugin.** Same reviewers, same checklist as any other code change.
- **Mandatory `plugin-validator` pass before merge.** The plugin-dev plugin ships a validator skill; run it in CI.
- **Standardized SKILL.md structure.** Frontmatter + sections + examples. Enforce via a lint rule.
- **Semver tags per plugin release.** Same versioning discipline as your service releases.
- **`CODEOWNERS` per plugin directory.** The team that owns the plugin reviews changes.
- **Security review for plugins that touch sensitive data.** Same gate as any production code.

Starter plugin ideas for ITSS-shaped workflows

- **KYC pattern checks** – detect missing onboarding fields, flagged jurisdictions, suspicious document types
- **AML pattern checks** – structuring detection, velocity rules, cross-account heuristics
- **Transfer-flow visualizers** – generate flow diagrams from transaction logs for audit
- **Fraud-rule explainers** – given a rule ID, produce a human-readable explanation + linked test cases
- **Regulatory regex linters** – match-and-flag patterns in customer comms for compliance
- **Card-issuance policy checks** – BIN range validation, network compatibility, jurisdictional restrictions
- **Internal architecture review skills** – canned prompts that map a microservice with your team's exact preferred output format

When a developer leaves, the plugin stays

This is the long-term value. The plugin is in the repo. The repo has owners. The reviewer set is the team. The validator catches drift. **Institutional capital that survives turnover.**

Diana's 12 Challenges → Workshop Landing

This table maps each item from the pre-workshop intake to where it lands today, the take-home artifact, and any follow-up out of scope for half a day.

#	Challenge	Where it lands today	Take-home artifact / Follow-up
1	Enterprise AI integration (auth platforms)	Theme 1 governance + MCP cheatsheet	MCP Supply Chain Checklist (handout) / Follow-up: MCP integration deep-dive
2	Documentation analysis	Theme 1 – Documentation Mindmap Prompt	`05_Documentation_Mindmap_Prompt.pdf` (this Drive folder)
3	Frontend TDD	Demo 2 Prompt 2 baseline + post-impl Playwright (BEFORE and AFTER)	Frontend testing pattern + Demo 2 paste-in (this handout)
4	Security / control over AI on client repos	Theme 1 Permissions / Sandbox primitive (four configuration surfaces)	MCP Supply Chain Checklist / Follow-up: security review with security lead
5	Responsible AI	Theme 1 governance + Brownfield kill signals	Kill-Signal Decision Card
6	Task decomposition	Demo 2 Prompt 2 Phase 2 PLAN (`CONFIDENCE_PLAN.md`)	Expected-plan outline pattern (this handout)
7	Iterative work	Demo 2 phase-based methodology (one superpowers prompt, six phases)	Phase-based loop section (above) + Demo 2 paste-in
8	Maturity in deciding when AI is appropriate	Theme 3 two-category diagnostic (CODE vs PEOPLE)	Kill-Signal Decision Card
9	Internal best practices	CLAUDE.md / AGENTS.md templates	`CLAUDE_AGENTS_md_Cheatsheet.md`
10	Underused testing automation	Demo 2 Prompt 2 mandatory before+after Playwright + AI Code Review Checklist take-home	Frontend testing + AI Code Review Checklist + Demo 2 paste-in
11	Anthropic / Claude Code focus	All themes (Claude Code, Superpowers, Plugins, MCP)	Starter pack install instructions (above)
12	Managerial adoption gap	Theme 3 90-day rollout	**Manager Day-7 Worksheet** (cheatsheet, new)

> ***Reading guide:*** *Items 1, 4, 5, 9, 11 land in the Governance bucket; Items 2, 3, 6, 7, 10 in Delivery methodology; Items 8 and 12 in Brownfield quality + adoption. The deck repeats this mapping at Slide 18A as a closing artifact.*

THE MEMORY PRIMITIVE – FOUR LAYERS

The Claude Code memory system is a structurally distinct primitive from skills (on-demand) and tools (actions). Four layers, all native to Claude Code:

1. **CLAUDE.md / AGENTS.md** – user-written persistent instructions, loaded at every session start. CLAUDE.md is the Claude-Code-specific variant; AGENTS.md is the vendor-neutral standard supported by Codex CLI, Cursor, GitHub Copilot, Gemini CLI, and Aider. Target under 200 lines per file; longer files consume context and reduce adherence.
2. **Auto Memory** – Claude-written learned patterns accumulating across sessions. Claude saves notes for itself as it works: build commands, debugging insights, architecture notes, code-style preferences, workflow habits. Distinguished from CLAUDE.md by source: CLAUDE.md is intentional and editorial; Auto Memory is learned and accumulative.
3. **Session Memory** – continuity within a session. The conversation history plus tool results plus loaded files. Lives inside the context window.
4. **Auto Dream** – background memory consolidation (Anthropic announced 2026-05-06). A scheduled process that reviews recent sessions and the memory store, identifies recurring mistakes and convergent workflows, and writes consolidated notes back into long-term memory. Self-improvement between runs without manual retraining.

Source: code.claude.com/docs/en/memory.

Open-set note: “The primitives” is not a closed list. Memory and Permissions / Sandbox are the additions we name explicitly today; others (LSPs, observability/event push) sit on the boundary between primitives and control layers depending on the taxonomy you adopt. Hooks come off the boundary — they’re Surface 3 of the Permissions / Sandbox primitive’s four config surfaces.

THE PHASE-BASED AGENTIC DELIVERY WORKFLOW (Superpowers)

*Term: a **subagent** is a fresh, isolated Claude Code worker the orchestrator dispatches via the Task tool. It has its own context window, no memory of prior subagent work, and returns a structured report. The orchestrator coordinates 2-5 subagents in parallel and synthesizes their outputs. This is how “parallel exploration” gets done inside a single Claude Code session.*

1. **RESEARCH** – Ask questions, don’t jump into code. The brainstorming skill writes the design as `spec.md` (committed to `docs/superpowers/specs/YYYY-MM-DD-<topic>-design.md`). “Do not start work yet.”
2. **PLAN** – File-level decomposition. Tasks of 2-5 min each. Checkpoint: approve the plan.
3. **EXECUTE** – One subagent per task. Orchestrator coordinates, never implements.
4. **REVIEW** – Spec compliance + code quality. Two separate subagents. Then human review.
5. **VERIFY** – Run ALL tests. Check against original spec. 3 random regression features.
6. **SHIP** – PR via GitHub MCP, linked to Jira, Slack notification. Clean up worktree.

The critical prompt: *“Do not start work yet. Let me know if you have any questions.”*

Worked Phase 5 example – Spring Boot reconciliation service

You just finished a feature: a new exception handler for `DataAccessException` in the reconciliation pipeline. The agent says “done.”

What Phase 5 VERIFY actually requires before you accept “done”:

1. Run **all** tests, not just the new ones: `./gradlew :reconciliation-service:test --rerun-tasks`. If any pre-existing test fails, the work is NOT done – regression.
2. Spec compliance check: re-read the original ticket. Did the agent solve what was asked, or what it interpreted? Skill `verification-before-completion` makes the agent write evidence

inline.

3. Working tree cleanup: `git status` should show only the files the agent claimed. Lock files, screenshots, leftover `*.tmp` – agents leave these. Delete before PR.
4. **3-random-regression**: pick 3 features completed in the last 2 weeks. Run their tests. Catches cross-contamination – the new exception handler may have shifted shared state.
5. Evidence in PR: paste the test command, the pass count, and a 1-line note for each previously-failing test that now passes (or stays passing).

The output is `code_review.md` (auto-generated by `requesting-code-review` skill) plus your verification log. Both committed; PM and QA read the artifacts, don't interrogate the dev.

CLAUDE.md TEMPLATE – Adapt for Your Project

Design principle: failure-mode-driven, not preferences-driven. Every rule answers: *"what mistake does this prevent?"* CLAUDE.md is not a wishlist; it's a behavioral contract that closes specific failure modes your team has observed in code review. If a rule sounds like "we prefer X", rewrite it as "we've seen Y go wrong, so never X."

```
## Project
Spring Boot 3.x [service name]. Production system.
Java 21, Gradle, PostgreSQL 16.

## Architecture
- Controller -> Service -> Repository (strict layers)
- Package: com.itss.[domain].{controller,service,repository,model,config}
- DTOs for API boundaries, entities for persistence. Never expose entities.

## Forbidden Patterns
- Never expose PII in logs (mask card numbers, account IDs)
- Always use parameterized queries (no string concatenation in SQL)
- No @Autowired on fields -- constructor injection only
- No business logic in controllers
- [Add your most common code review comments here]

## Mistake Journal
- [Date]: Claude did X, should have done Y. Root cause: missing context about Z.

## Testing
- Framework: JUnit 5 + Mockito + Testcontainers
- Run: ./gradlew test
- Every public service method has a test
```

Rules: Keep under 200 lines. Commit to repo. Review in PRs. Split overflow into `.claude/rules/*.md`.

What belongs here: project architecture, forbidden patterns, test commands, sensitive-data rules, and the mistake journal. If a rule must be enforced deterministically, mirror it as a hook or sandbox rule; CLAUDE.md is guidance loaded into context, not a security boundary.

Large-codebase rule: launch Claude Code from the package/service directory you are changing. Claude walks upward and loads relevant parent `CLAUDE.md` files, so monorepo context is preserved without drowning the task in unrelated rules. Use `.claude/rules/*.md` for scoped domains; use `claudeMdExcludes` when a parent memory file is irrelevant or contradictory.

Feature placement rule: recurring context goes in `CLAUDE.md`; deterministic lifecycle checks go in hooks/hookify; external system access goes through MCP; independent parallel work gets a subagent, worktree, or agent team.

AGENTS.md TEMPLATE – Multi-Agent Orchestration Context

CLAUDE.md is for the agent doing the work. AGENTS.md is for *teams* of agents and for cross-tool compatibility. Codex reads `AGENTS.md` directly; Claude Code does not.

Important: Claude Code does NOT read AGENTS.md natively - it only reads CLAUDE.md. To make AGENTS.md active in a Claude Code session, **import it from CLAUDE.md** by adding a line like `See @AGENTS.md for multi-agent orchestration.` (or symlink `AGENTS.md → CLAUDE.md`).

```
## Roles
- Lead agent: planning + orchestration; never implements
- Implementer subagents: one per task, fresh context, no inheritance
- Reviewer subagents: spec compliance + code quality (separate agents)
- QA fleet: pr-review-toolkit (6 agents) before human review

## Handoff Protocol
- Lead -> Implementer: task packet (Goal + Files + Forbidden + Success Criteria)
- Implementer -> Reviewer: diff + tests + claim of done. No implementation context
inherited.
- Reviewer -> Lead: severity-ranked findings with file:line, NOT free-form prose
- QA -> Human: GO / NO-GO with reproduction evidence

## Shared Conventions
- Read CLAUDE.md FIRST. AGENTS.md augments, never contradicts.
- Run kill signal classification (GREEN/YELLOW/RED) on every task before starting
- Forbidden Patterns from CLAUDE.md are inherited by all subagents
- Every subagent runs in its own git worktree (isolation, parallelism)
- 3-random-features regression: pick the next feature + 3 completed features, run their
tests first

## Test Authority
- Implementer MUST run tests before claiming done
- Reviewer MUST run independent verification (not the implementer's tests)
- Final QA fleet runs full regression + 3 random completed features

## Cross-Tool Compatibility
Codex reads AGENTS.md directly.
For Claude Code: import via `@AGENTS.md` inside CLAUDE.md OR symlink AGENTS.md ->
CLAUDE.md.
For GitHub Copilot: official repo instructions live in `.github/copilot-
instructions.md`; mirror only the essential AGENTS.md rules there if Copilot is in
scope.
Tool-specific notes belong in `.claude/`, `.cursor/`, `.aider/`. Never collide.
```

Rules: Same as CLAUDE.md – under 200 lines, committed, PR-reviewed. Use when you orchestrate 2+ agents on a single feature.

What belongs here: agent roles, handoff protocol, worktree isolation, review gates, and test authority. Do not copy the whole CLAUDE.md into AGENTS.md. Import or point to it, then keep AGENTS.md focused on orchestration.

GOVERNANCE – FOUR SURFACES

The Permissions / Sandbox primitive has four configuration surfaces; PocketOS happened because three of the four surfaces and the adjacent secrets concern were absent.

> ***Honest scope note (from the May 28 session):*** *Surface 1 was the only surface built live on the whiteboard during the exercise – granular Allow/Ask/Deny overrides on top of auto mode. Surfaces 2-4 (project settings, hooks, OS sandbox) plus the adjacent concerns (secrets management, security-guidance plugin) were described on stage but each is its own 1-2 week build with your security lead; baselines + starter JSON are below.* ***Plan for 90-day construction, not one-afternoon adoption.***

Surface 1: Claude Code defaults (Allow/Ask/Deny rules) – start with `permissions.defaultMode: "auto"` as the default (the key lives inside the `permissions` block, not at the top level - easy to get wrong by hand). Claude's classifier decides per-action whether to ask, based on risk and context. For most repos, auto handles 80-90% of the decisions correctly with far less friction than approving each file read manually.

Allow / Ask / Deny rules are granular overrides on top of auto - declarative exceptions for cases where the classifier is wrong, or where compliance requires explicit declarative rules. Most teams need 10-15 override rules per repo, not dozens.

Settings.json scopes (most-restrictive first): **managed > local > project > user**. Evaluation order within scope: **Deny → Ask → Allow**, first match wins; unmatched defaults to `auto`.

DENY override (hard block)	ASK override (always approve)	ALLOW override (skip auto's question)
Delete production data	Install new packages	Read source files (if auto frictions)
Access .env secrets*	Write config files	Run test suite
Push to main directly	Execute shell commands	Format/lint code
DROP/TRUNCATE/DELETE without WHERE	Create PRs	Write to /src and /tests
Modify CI/CD pipelines	Post to Slack channels	Read Jira tickets (MCP)
Access PII without authorization	Run database queries	Git diff, git status

*Deny on `Read(./.env)` alone is insufficient as a hard boundary - permission rules have had documented bypass classes (arbitrary subprocess access, parser-cap variants in early 2026 - some patched). See Surface 4 for the technical boundary: OS-level sandbox `denyRead`.

Rule of thumb: if your ALLOW list is growing past 10 items, your CLAUDE.md probably needs to be more explicit instead - auto will be smarter with better context.

Surface 2: Project settings – `.claude/settings.json` committed to repo. Your team's specific Allow/Ask/Deny overrides live here, versioned alongside the code they govern. PR review applies. Onboarding becomes "clone the repo, you have the rules."

Surface 3: Hookify rules (programmable hooks) – `.claude/hookify.<rule-name>.local.md`, one rule per file, markdown with YAML frontmatter, enforced via PreToolUse / PostToolUse hooks. Programmatic gates the agent CANNOT bypass: refuse `rm -rf` regardless of path, block writing a `.tsx` component file without a sibling test, warn on `console.log`, block edits to `.env*`.

Unlike Surface 1/2 which are declarative rules, Surface 3 is code your team writes to enforce non-negotiables deterministically.

Hookify cheatsheet (the schema you'll actually author against):

- **Rule file location:** `.claude/hookify.<rule-name>.local.md` – versionable in git, one rule per file.
- **Frontmatter fields:**
  - `name` – rule id
  - `enabled` – `true` | `false`
  - `event` – `bash` | `file` | `stop` | `prompt` | `all`
  - `pattern` (simple form, single regex) OR `conditions` (advanced form, list of `field` / `operator` / `pattern` triples)
  - `action` – `warn` (show message, allow) | `block` (prevent the operation). Default `warn`.
- **Fields available per event:**
  - `bash` -> `command`
  - `file` -> `file_path` | `new_text` | `old_text` | `content`
  - `prompt` -> `user_prompt`
  - `stop` -> `transcript`
- **Operators:** `regex_match` | `contains` | `equals` | `not_contains` | `starts_with` | `ends_with`
- **Rules are active immediately – no restart needed.**

Concrete example – `test-first-for-tsx` (the rule Mihai authored live in Demo 1 Part 4). Blocks creating a `.tsx` file whose path doesn't contain `.test.` – forcing the test file to be written first. This is the real-schema replacement for the older "block commit without tests" framing: hookify's `bash` event sees only the command string, not git's staged-file state, so the enforceable point is the **file write**, not the commit.

```
---
name: test-first-for-tsx
enabled: true
event: file
action: block
conditions:
  - field: file_path
    operator: regex_match
    pattern: \.tsx$
  - field: file_path
    operator: not_contains
    pattern: .test.
---
```

Body of the rule (markdown after the frontmatter) is the message Claude sees when the rule fires – explain why, list the recovery steps.

Surface 4: OS sandbox (Oct 2025, kernel-enforced) – bubblewrap on Linux/WSL2, Seatbelt on macOS, restricted tokens on Windows. Two boundaries: filesystem + network. The hard boundary. Enable in managed settings:

```
{"sandbox":{"enabled":true,"failIfUnavailable":true,
 "filesystem":{"denyRead":["~/.aws/**","~/.ssh/**","./.env*","./secrets/**"]},
 "network":{"allowedDomains":["github.com","*.npmjs.org","api.anthropic.com"]}}}
```

Start: `/sandbox` inside a session. Per the Anthropic Oct 2025 launch announcement, sandbox-on yields a substantial reduction in permission prompts - figure is vendor self-report, not independent measurement; treat the direction as the claim, not the precise percentage.

Why Surface 4 is the hard boundary: Surfaces 1-3 are agent-level (declarative rules + programmable hooks). All of them can theoretically be tricked by prompt injection — the agent reads attacker content, gets convinced to escalate, the gate is bypassed. Surface 4 is the kernel itself. Kernel calls cannot be tricked by prompt injection. If the kernel refuses, the kernel refuses. End of story.

Adjacent concern: Secrets management (minimum for PSD2 / PCI-DSS):

- Managed `sandbox.filesystem.denyRead` (Surface 4) covers `.env*`, `~/.aws/**`, `~/.ssh/**`, `./secrets/**` (OS-level, can't be shell-bypassed)
- PreToolUse hook (Surface 3) on Bash matching `cat|grep|less|head|tail` against credential regex
- No `.env` files on disk during Claude sessions - use 1Password Environments, HashiCorp Vault, or AWS Secrets Manager
- MCP servers never store plaintext credentials in config - use OS keychain or Vault
- ZDR (Zero Data Retention) addendum required - verify in contract before first production use

Secrets management is *adjacent* to the four configuration surfaces — it's about WHERE you store sensitive values, which is a related but distinct concern from gating agent actions.

Adjacent concern: `security-guidance` plugin – Anthropic-verified plugin that catches 8 vulnerability categories before code is written, including command injection in GitHub Actions, unsafe `child_process.exec`, eval/Function, XSS via `innerHTML`, Python pickle deserialization, and `os.system`. Technically uses the Surface 3 hooks mechanism (PreToolUse gates) but distributed as a plugin so you don't have to write the patterns yourself.

Bonus tool - visualization (`understand-anything`): Open-source MIT plugin (`Lum1104/Understand-Anything` marketplace) that builds an interactive knowledge graph for any codebase. Two views: Structural (file/function/class graph for developers) and Domain (business domains with extracted business rules and entities - readable by managers and PMs without code). Not a governance surface; a comprehension layer. Demo 1 Step P2.5 showed this live on Codex. See the Understand_Anything_Quickref cheatsheet for install + commands + troubleshooting.

Why all four surfaces plus the adjacent concerns: PocketOS (Apr 2026) had Surface 1 (defaults) but no Surface 4 (OS sandbox), no Surface 3 hooks (no destructive-op block), and the Railway API token was on disk instead of in a vault (adjacent secrets concern missing). 9 seconds, production DB + same-volume backups gone. The system prompt said "NEVER GUESS." The agent guessed. One surface of configuration is not governance.

Governance is also positive – build before you restrict. The four layers above are guardrails. The other half of governance is what you BUILD inside them. Skills and plugins

authored by your team, versioned in git, distributed via a **company-private plugin marketplace**, reviewed in PR like any other code. Demo 1 Part 2 walked the live ritual: the `plugin-dev` SDK scaffolds a complete marketplace + sample plugin in about a minute, and what you get is **a git repo with a manifest**. Not a special distribution channel. Not an opaque artifact. A repo your team owns and maintains exactly like any other – PR review, semver tags, `CODEOWNERS`, security review when the plugin touches sensitive data. The “Build & Own Your Marketplace” section earlier in this handout has the starter plugin ideas for ITSS-shaped workflows (KYC patterns, AML pattern checks, transfer-flow visualizers, fraud-rule explainers, regulatory regex linters). Distribute via `/plugin install <name>@<your-private-marketplace>`. **Governance through construction, not just through restriction.** When a developer leaves, the plugin stays. That is institutional capital.

BROWNFIELD DECISION FRAMEWORK

Apply this **before** assigning any AI-heavy task on a legacy codebase. Three steps.

Kill Signal Decision Card — CODE / PEOPLE Triage

Step 1: List the signals you see on the project.

Use the 8-signal checklist (full list in `02_Kill_Signal_Decision_Card.pdf` in this Drive folder):

1. No Tests (<30% coverage)
2. No Documentation (tribal knowledge only)
3. Tight Coupling (>5 modules)
4. Complex Business Rules (scattered, undocumented)
5. Regulatory / Compliance Constraints
6. Team Can’t Evaluate Output
7. Model-Context Fit (thin pretraining)
8. Velocity-of-Change (framework mid-migration)

Step 2: Tag each signal — CODE or PEOPLE.

CODE signals = what’s missing is an artifact AI can produce: Signals 1, 3, 4, 5, 7, 8.

PEOPLE signals = what’s missing is human knowledge or evaluation skill: Signals 2, 6.

Step 3: For each CODE signal, name the AI pre-work. For each PEOPLE signal, name the conversation or mentorship investment.

Signal	If CODE	If PEOPLE
1 No Tests	Characterization tests generated	—
2 No Documentation	—	Conversation with the senior + document
3 Tight Coupling	Dependency graph + seam extraction	—
4 Complex Rules	Golden tests + rule registry	—
5 Regulatory	Lint rules + compliance hooks	—
6 Can’t Evaluate	—	Senior mentorship + paired review
7 Model-Context	Domain README + DSL grammar notes	—
8 Velocity-of-Change	Migration tests + version-freeze	—

Step 4: Count by category. Read the traffic light:

- 0-1 signals: GREEN. Go.
- 2-3 signals: YELLOW. Count by category. CODE-heavy YELLOW: AI pre-work plan. PEOPLE-heavy YELLOW: hiring or mentorship plan.
- 4+ signals: RED. Invest in codebase readiness first.

Diagnostic question (the take-home): “Is what’s missing an artifact, or human knowledge?”

Recent Claude Code brownfield failure modes to watch (May 2026):

Failure mode	What happens	When it appears	Why it happens	How to mitigate
Wrong-scope implementation	Agent edits the wrong layer, duplicates existing logic, or fixes the visible symptom.	Vague prompt in a large repo; no subsystem map; implicit legacy convention.	Claude can search live files, but it needs starting context to know where to look.	Use plan mode first; point to likely files; require codebase citations; start from a failing test.
Context / compaction drift	Session forgets constraints, repeats itself, or contradicts earlier choices.	Long debugging session; stale resume; kitchen-sink session; noisy tool output.	Context fills quickly, compaction is lossy, and old history distracts the next task.	Use shorter sessions, `/context`, focused `/compact`, `/clear` between tasks, and subagents for noisy exploration.
Verification theater	Patch looks finished but no test, screenshot, log, or deterministic check proves it.	Weak tests, UI-only change, hidden business rule, or agent-written happy-path tests.	The task rewards plausible completion unless a verifier is part of the prompt.	Require unit/integration tests, Playwright evidence, build logs, or reviewer passes before merge.
Auto-mode scope gap	Risky state change happens through an indirect path, config edit, or in-scope command.	Cleanup, config, migration, infra-state, or ambiguous environment-fix tasks.	Permission automation reduces prompts but is not complete blast-radius control.	Use plan/default mode for risky work; add deny rules and hooks; sandbox; keep destructive commands manual.
Production side effect	Agent reaches real cloud, database, or admin APIs and changes state.	Dev shell contains production tokens, cloud CLIs, database URLs, or admin API credentials.	Actual credentials and network access define damage, not the prompt text.	Use least-privilege dev tokens, prod/dev separation, human approval for deletes/migrations, and offline backups.
Dependency hallucination / slopsquatting	Fake or untrusted npm/PyPI package enters the plan or lockfile.	New dependency request, migration, generated skill, or autonomous `npm` / `pip` install.	Frontier models still invent plausible package names that attackers can register.	Prefer lockfile-only installs; verify registry, maintainer, version, provenance, and existing alternatives.
Harness / model drift	Known workflow becomes unreliable after a model, CLI, prompt, permission, or cache change.	Demo, regulated workflow, long-lived playbook, or pinned prompt after a product release.	Agent quality depends on the whole harness, not just the model.	Pin demo versions; set model/effort explicitly; record fallback videos; review CLAUDE.md, hooks, plugins quarterly.

Primary sources, spot-checked May 21, 2026: Anthropic large-codebase guidance; Claude Code session-management guidance; Claude Code best practices, permission-mode docs, and sandboxing docs; Anthropic April 23 Claude Code postmortem; AmPermBench; package-hallucination replication; PocketOS incident coverage; and Claude Code releases for live-checking CLI, permission, plugin, and agent-session drift. Secondary architecture background: Claude Code from Source.

LLM Tells — Slop Detection in Code Review

When you review AI-generated code, four signals to spot slop:

1. **Excessively defensive code** — try-catch on every line, even where exceptions can't be thrown.
2. **Duplicated logic** — two implementations of the same functionality. One doesn't work, the second does, the first was left in the code.
3. **Test artifacts in working tree** — screenshots, lock files, test data forgotten in the working tree.
4. **Confident nonsense** — code that looks professional but does something completely wrong.

If you see them, don't accept. Reject slop.

(Source: Theme 3 — Brownfield Mastery, the LLM-Slop section. The four tells above are the participant-take-home subset; the underlying 2026 brownfield failure-mode taxonomy is captured in the kill-signal framework on `02_Kill_Signal_Decision_Card.pdf` in this Drive folder.)

Match the right tool to the work:

Task Type	Best Tool	Why
Architecture review + codebase exploration (any language)	Claude Code	Deep file reading + agent loop
Spring Boot refactoring on GREEN/YELLOW projects	Claude Code + Superpowers	Methodology enforcement (phase-based)
Code review at PR scale	pr-review-toolkit (6 local agents)	Specialized agents, parallel review
Critical-path review before merge	`/ultrareview` (cloud fleet, if data policy allows)	Independent verification, parallel multi-agent review
Long-running migration (multi-day)	Claude Code Agent View (`--bg`)	Async, 2-4 parallel sessions
Quick inline autocomplete	GitHub Copilot or Cursor Tab	Lower friction for trivial edits
Pair-programming UI session	Cursor or Aider	UI-first ergonomics
Documentation analysis (non-code)	CoWork (separate licensing)	Same model family, non-dev surface

Rule of thumb: Match tool to task, not task to tool. Same project may use 3 different tools across its lifecycle.

TAKE-HOME PROMPT: Map Any Repo's Architecture into an Interactive HTML

You saw this in Demo 1 - a live architecture review of two coding agents (Codex CLI and anomalyco/opencode) explored side-by-side, each generating its own architecture HTML as the byproduct. The same prompt + parallel-pane pattern is the repo-agnostic Monday recipe - generic version in the `Repo_Architecture_Mapping_Prompt` cheatsheet (separate handout).

Two ways to run it. (1) Paste the prompt from the cheatsheet into any Claude Code session – the verbose path, useful when you want to tweak the scopes. (2) Install the `architecture-html` plugin from your internal marketplace (the one scaffolded live in Demo 1 Part 2 – same prompt encoded as a reusable skill) and run `architecture-html` as a one-word invocation on any repo. The plugin is the productionized form; the cheatsheet is the source.

TL;DR - the prompt does this:

1. Reads top-level structure (manifest files, README) - ~2 min.
2. Decomposes the repo into 3–5 exploration scopes (entry points / business logic / data / integrations / infrastructure - adapts to what's actually there).
3. Dispatches one subagent per scope via the Task tool, in parallel. Each returns a structured module-by-module report with file:line citations.
4. Synthesizes all subagent outputs into a single coherent architecture model.
5. Generates `./docs/architecture-map.html` - color-coded sections, clickable `file://` links to actual source files, "what's uncharted" footer, Open Questions surfaced.
6. Validates every link before claiming done.

Pairs with `/goal`:

```
/goal Map this repo's architecture into an interactive HTML using parallel subagents, with
file:line citations for every claim and clickable links to the actual source files.
```

Then paste the full prompt body from the cheatsheet.

When to use it:

- A legacy repo nobody on the team fully remembers
- A microservice you're about to inherit
- An open-source library you're vendor-evaluating
- A monorepo where you want a high-level map for the docs site

Wall-clock time: ~8–10 minutes for an average-sized repo. Run it overnight or during a meeting if it's a big monorepo.

Browser: Chrome blocks file:// → file:// navigation by default - launch Chrome with `--allow-file-access-from-files --user-data-dir=/tmp/chrome-workshop` to enable clickthrough into source files. The `/tmp/` profile isolates the security-relaxed session from your normal Chrome browsing. We use this exact wrapper on the workshop laptop (see your starter pack).

TAKE-HOME PLUGIN: Understand Anything for Interactive Dashboards

You saw this in Demo 1 Step P2.5 - the polished alternative architecture view (Structural + Domain) over the same Codex codebase. The Domain view turns code into business domains

readable by managers. The plugin is open-source MIT, runs locally, no data leaves your machine.

Install (one-time, inside a Claude Code session):

```
/plugin marketplace add Lum1104/Understand-Anything
/plugin install understand-anything
```

Generate dashboard for any repo (3-5 min first time, incremental after):

```
cd /path/to/your/repo
claude
```

Then inside the Claude Code session:

```
/understand
/understand-dashboard
```

Two views, two audiences:

- **Structural view** - file/function/class nodes color-coded by architectural layer, dependency arrows, complexity labels. For developers.
- **Domain view** - business domains (e.g., “Payment Processing”), business rules extracted from code, entities, cross-domain flows. For managers and PMs reviewing the work.

Click any node for a detail panel with plain-English description, tags, business rules, and related components.

When to use it:

- **Onboarding** a new developer to a legacy repo (~5 min visual map beats 3 weeks of code reading)
- **PR / impact review** - `/understand-diff` shows which graph nodes a PR touches and what depends on them
- **Architecture stakeholder reviews** - Domain view lets non-engineering stakeholders read the code’s structure without reading code

See the full reference in the **Understand_Anything_Quickref** cheatsheet.

AI PRODUCTIVITY – Where It Actually Helps

Task Type	Gain	Notes
Migrations / upgrades	Very high	Stripe: 10K lines in 4 days vs. 10 engineer-weeks
Boilerplate (controllers, DTOs)	Very high	Repetitive patterns = agent strength
Test writing	High	Agent generates, human validates
Codebase navigation	High	Agent reads 100K+ LOC faster than you
Refactoring	Medium-high	Needs good CLAUDE.md
Complex business logic	Medium	Agent misses domain nuances
Security-sensitive code	Low	Always human-led

Honest math: 1h of Claude work = 2-3h of human validation. 10h task becomes ~4h.

AI CODE REVIEW CHECKLIST – Java / Spring / Banking

Apply **before** merging any AI-generated PR. Each item maps to OWASP Top 10 or banking compliance. Items prefixed with [pr-rt] are auto-checked by pr-review-toolkit; the rest stay human-owned.

Security (OWASP-aligned):

- ☐ No string concatenation in SQL/JPQL/HQL - parameterized queries only (OWASP A03)
- ☐ No PII in logs - mask IBAN, card numbers, account IDs (PCI-DSS, GDPR)
- ☐ No secrets in code - `@Value` from environment, never hardcoded (OWASP A05)
- ☐ Input validation on every controller - `@Valid` + custom validators (OWASP A03)
- ☐ Authorization on every endpoint - `@PreAuthorize` at method level (OWASP A01)
- ☐ No unsafe deserialization - no Jackson default typing without allowlist (OWASP A08)
- ☐ XXE prevention - `DocumentBuilderFactory.setFeature(FEATURE_SECURE_PROCESSING, true)` (OWASP A05)
- ☐ No SSRF - outbound URL validation against allowlist (OWASP A10)

Exception handling [pr-rt: Silent Failure Hunter]:

- ☐ No empty catch blocks - log + rethrow OR handle specifically
- ☐ No catching `Exception` / `Throwable` broadly - catch specific types
- ☐ No swallowing `DataAccessException` without compensating action
- ☐ `@Transactional` rollback rules explicit (`rollbackFor = Exception.class` for runtime + checked)
- ☐ No “log and continue” on critical paths (transfers, reconciliation, audit)

Spring-specific:

- ☐ Constructor injection only - no `@Autowired` on fields
- ☐ `@Service` for stateless logic, `@Component` for utility, `@Repository` for data
- ☐ DTOs at controller boundary - entities never exposed

- [ ] `@Transactional` placement: service layer only, NOT controller
- [ ] Lazy-loading hazards: no `@ManyToMany` in DTO mapping without `@Fetch(FetchMode.JOIN)`
- [ ] No business logic in `@RestController` - controllers thin, services thick

Testing [pr-rt: PR Test Analyzer]:

- [ ] Every public service method has unit test
- [ ] Repository tests use `@DataJpaTest` or Testcontainers (NOT `@SpringBootTest` unless integration)
- [ ] Mock external services - no real HTTP/DB in unit tests
- [ ] Coverage gates: new code ≥80% branch coverage
- [ ] Edge cases: null inputs, empty collections, max boundary, negative amounts

Banking-specific:

- [ ] Decimal arithmetic uses `BigDecimal` - never `float`/`double` for money
- [ ] Currency conversion has source-of-truth - no hardcoded rates
- [ ] Audit log writes are non-cancellable - separate transaction or outbox pattern
- [ ] Idempotency keys on all state-changing endpoints (transfers, payments)
- [ ] Rate-limit on transaction endpoints (Bucket4j or Spring Cloud Gateway)
- [ ] PII access logged separately for compliance audit trail

AI-specific gate:

- [ ] PR description includes which agent generated/reviewed the code
- [ ] Human reviewer ran the test suite locally (not just CI)
- [ ] No commented-out code or “TODO” markers left by the agent
- [ ] Forbidden Patterns from CLAUDE.md verified absent
- [ ] If AI-generated method >50 lines: human re-derived the logic mentally before approving

Process: Run pr-review-toolkit first (catches the `[pr-rt]` rows). Human takes the rest. `/ultrareview` is an optional cloud review for critical-path PRs only when your data policy allows remote sandbox upload; it is not available with ZDR or Bedrock/Vertex/Foundry deployments.

90-DAY ROLLOUT – Adapt to Your Reality

Days 1-30 (Foundation):

- Week 1: 3-4 dev champions install starter pack (5 plugins, incl. one-time `/plugin marketplace add Lum1104/Understand-Anything` for understand-anything). One GREEN task each + run `/understand-dashboard` on that repo for visual onboarding. 1 manager runs kill signal assessment on a candidate project.
- Week 2: Write CLAUDE.md for one project. QA tries pr-review-toolkit on one PR.
- Week 3-4: Share results in standup. Agree on first Forbidden Patterns.

Days 31-60 (Expansion):

- Champions train remaining team (pair sessions, not lectures).
- Starter pack mandatory on all machines. Kill signal assessment on top 5 projects.

- First consolidated governance review: forbidden patterns confirmed, hook coverage assessed, mistake journal reviewed.

Days 61-90 (Standardization):

- CLAUDE.md maintenance rotation. Monthly review cycle.
- Full flow: PM structures requirements -> Dev (Superpowers) -> QA (pr-review-toolkit) -> PM updates stakeholders.
- Quarterly “What Changed” assessment.

MANAGER MONDAY CHECKLIST (the week after the workshop)

If you manage a team and you sat through this workshop, this is the one-page action list for the five working days that follow. Built so a manager can defend a pilot to leadership without re-reading the deck.

- ☐ **Run kill signal assessment on your top 3 active projects.** Use the Kill Signal Decision Card. 15 minutes per project. Output: one of GREEN / YELLOW / RED per project. This is what you walk into the leadership conversation with.
- ☐ **Identify your 3 candidate champions and your 2 likely skeptics.** Champions are self-selecting volunteers - not nominees. Skeptics matter because they tell you what the failure modes will be. Don’t ignore them.
- ☐ **Schedule a 1:1 procurement conversation with Diana or Mihai inside 10 working days.** Costs and licensing were deliberately not on stage; the 1:1 is where you get numbers calibrated to ITSS’s volume and timeline.
- ☐ **Block 2 hours on calendar for the first CLAUDE.md review PR.** Not for writing it - for reviewing what a champion drafts. That PR is your first governance artifact.
- ☐ **Pick ONE GREEN project for the pilot and write down what would kill it.** Three or four explicit kill criteria. Without kill criteria, “the pilot is ambiguous at day 90” is a failure mode in itself. Examples: “if no CLAUDE.md committed by day 30, kill”; “if security-guidance plugin not active by day 14, kill”; “if more than 2 developers express friction with auto mode by day 21, retune CLAUDE.md or kill.”

If you do these five things in week one, you walk into the day-30 review with a defensible position regardless of how the pilot is trending. If you skip any of them, day 30 becomes a debate about feelings instead of evidence.

PROCUREMENT NOTE

Licensing, pricing, and procurement specifics are intentionally handled off-stage. Contact Diana or Mihai directly for a 1:1 procurement conversation tailored to ITSS’s volume and timeline. Anthropic sales: https://claude.com/contact-sales. When you talk to them, ask for SSO, audit logs, EU data residency, managed plugin marketplace, and the ZDR addendum.

KEY COMMANDS

Command	What It Does	Context
`claude`	Launch Claude Code session	bash
`claude --bg [task]`	Launch background agent (Agent View)	bash
`/plugin marketplace add [repo]`	Add a custom marketplace (the official `claude-plugins-official` is already built-in)	inside Claude Code
`/plugin install [name]@[marketplace]`	Install a plugin	inside Claude Code
`/plugin list`	List installed plugins	inside Claude Code
`/superpowers`	Show Superpowers skills	inside Claude Code
`/hookify`	Create/manage team rules	inside Claude Code
`/ultrareview`	Cloud-based multi-agent code review; only if policy allows remote sandbox upload	inside Claude Code
`/review`	Local code review	inside Claude Code
`/dream`	Manual memory consolidation	inside Claude Code

BANKING-SPECIFIC CHECKLIST

- ☐ Request Zero-Data-Retention addendum from Anthropic before using on client code
- ☐ Enterprise plan for audit logs, SSO, data residency
- ☐ EU AI Act readiness: as of May 21, 2026, the Commission's May 7 Digital Omnibus agreement sets high-risk AI rules for key standalone areas from Dec 2, 2027 and product-integrated systems from Aug 2, 2028. Treat exact legal scope as a counsel-check item; document AI governance now regardless. Source: https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai
- ☐ Multi-tier code review (automated + human + audit log) = **evidence material** for the compliance conversation with legal – not compliance pre-built
- ☐ Financial-services adoption is no longer theoretical: Goldman Sachs is a founding partner in Anthropic's May 4, 2026 enterprise AI services firm, and Anthropic/AWS financial-services material cites Verisk and NBIM as real-world examples. Sources: https://www.nasdaq.com/press-release/anthropic-partners-blackstone-hellman-friedman-and-goldman-sachs-launch-enterprise-ai and https://resources.anthropic.com/ty-financial-services-ebook-anthropic-aws

Contact: Mihai Cvasnievschi | mihai.cvasnievschi@gmail.com **Methodology endures. Tools rotate. Frameworks of judgment are the real skill.**